

Lesson 17 Handout – REINFORCE with Baseline

Reinforcement Learning

DSOR 646

This handout shows a Python implementation of the REINFORCE with Baseline algorithm, a policy gradient solution procedure. The pseudocode for the algorithm is displayed on page 330 of the course textbook. A linear value function approximation (VFA) scheme, employing a tile coding approach, is applied to represent the value of state-action pairs. The algorithm determines a superlative policy and value function for the *CartPole-v1* Farama Foundation Gymnasium Environment. The provided code generates a graph displaying the *learning curve* and other important performance statistics.

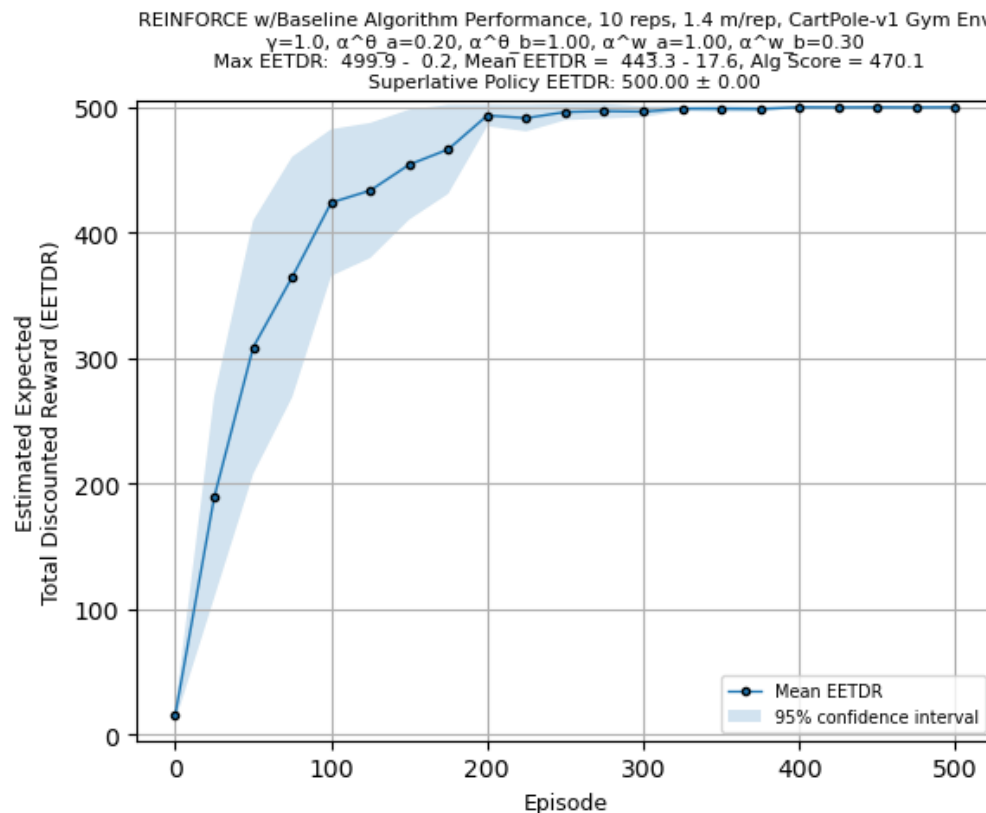


Figure 1: REINFORCE learning curve for the Farama Foundation Gymnasium *CartPole-v1* Environment.

```
1 """
2 DSOR 646 - Reinforcement Learning
3 Dr. Matthew Robbins
4 Lesson 17 Handout -- REINFORCE with Baseline
5 Farama Foundation Gymnasium CartPole-v1 Environment
6 https://gymnasium.farama.org/environments/classic_control/cart_pole/
7 """
8
9 # import required package libraries
10 import gymnasium as gym
```

```

11 import numpy as np
12 import time
13 from scipy.stats import t
14 import matplotlib.pyplot as plt
15 from sklearn import metrics
16 from tiles3 import tiles, IHT
17 import collections
18 import copy
19
20 # create Farama Foundation Gymnasium environment
21 envName = "CartPole-v1"
22 env = gym.make(envName)
23
24 print(f"\nFarama Foundation Gymnasium version: {gym.__version__}")
25 print(f"Environment: {envName}")
26 # Observation and action space
27 print("Observation space: {}".format(env.observation_space))
28 print("Action space: {}".format(env.action_space))
29
30 """ Algorithm Hyperparameters """
31 # algorithm name
32 algName = "REINFORCE w/Baseline"
33
34 # PFA/VFA tile coding scheme for state-action variables
35 # max size of integer hash table (iht)
36 # too small will impact performance
37 # too large will impact computational time
38 max_size_iht = 2**11 # 11
39 # size parameters for tilings and tiles per tiling
40 num_tilings = 4 # 4 partitions
41 num_tiles_per_tiling = 4 # 4 elements per partition
42
43 # PFA coefficient smoothing rule
44 alpha_theta_a = 0.2 # (*) # 0.05
45 alpha_theta_b = 1.0 # (*) # 1.0
46 def alpha_theta(n):
47     return (alpha_theta_a/(1+n)**alpha_theta_b)/num_tilings
48
49 # VFA coefficient smoothing rule
50 alpha_w_a = 1.0 # (*) # 1.0
51 alpha_w_b = 0.3 # (*) # 0.5
52 def alpha_w(n):
53     return (alpha_w_a/(1+n)**alpha_w_b)/num_tilings
54
55 # discount factor
56 gamma = 1.0
57
58 # number of algorithm runs
59 Z = 10
60
61 # number of episodes
62 M = int(0.5e3)
63
64 # policy evaluation test frequency
65 test_freq = 25
66
67 # number of replications per test
68 num_test_reps = 30
69
70 # random number generator seed manual offset
71 offset = 0
72
73 # dimensionality of state space
74 num_state_dims = len(env.observation_space.low)

```

```

75
76 # discrete action space -- number of actions
77 num_actions = env.action_space.n
78
79 # define reference points in state space for use with PFA
80 # Slow = np.array(env.observation_space.low)
81 # Shigh = np.array(env.observation_space.high)
82 # overwrite environment infinite boundaries with realistic finite ones
83 Slow = np.array([-2.5, -4.0, -0.3, -1.9])
84 Shigh = np.array([2.5, 3.3, 0.3, 3.9])
85 Srange = Shigh-Slow
86 scaleFactor = num_tiles_per_tiling/Srange
87
88 # basis function reports active tiles for (s,a) pair
89 def x_indices(s,a,iht): # (i.e., x(s,a))
90     return tiles(iht,num_tilings,list(s*scaleFactor),[int(a)])
91
92 # action preference function
93 def h(s,a,theta,iht):
94     hhat = 0
95     tiles = x_indices(s,a,iht)
96     for tile in tiles:
97         hhat += theta[tile]
98     return hhat
99
100 # policy function approximation (e.g., Boltzmann, exponential soft-max)
101 def pidist(s,theta,iht):
102     denom = np.sum([np.exp(h(s,b,theta,iht)) for b in range(num_actions)])
103     return np.array([np.exp(h(s,a,theta,iht)) for a in range(num_actions)]/denom)
104
105 # policy function approximation (e.g., Boltzmann, exponential soft-max)
106 def greedypi(s,theta,iht):
107     return np.argmax(pidist(s,theta,iht))
108
109 # stochastic PFA
110 def pi(s,theta,iht):
111     return np.random.choice(num_actions,1,p=pidist(s,theta,iht).ravel())[0]
112
113 # basis function reports active tiles for state
114 def phi_indices(s,iht):
115     return tiles(iht,num_tilings,list(s*scaleFactor))
116
117 # approximate Q-function
118 def Vhat(s,w,iht):
119     vhat = 0
120     tiles = phi_indices(s,iht)
121     for tile in tiles:
122         vhat += w[tile]
123     return vhat[0]
124
125 # function for computing confidence interval
126 def confinterval(data,alpha=0.05):
127     n = np.size(data) # number of data points
128     se = np.std(data,ddof=1)/np.sqrt(n) # standard error
129     ts = t.ppf(1-alpha/2, n-1) # t-score for two-tail test
130     mean = np.mean(data)
131     halfwidth = ts*se
132     return mean, halfwidth
133
134 # define policy evaluation function
135 def evaluate_policy(theta, iht, num_reps, seed_mult=1):
136     # initialize test data structure
137     test_data = np.zeros((num_reps))
138     # run num_test_reps replications per test

```

```

139     for rep in range(num_reps):
140         # initialize episode complete flags
141         terminated = False
142         truncated = False
143         # initialize episode reward
144         Gtest = 0
145         # initialize the system by resetting the environment, obtain state var
146         state = env.reset(seed=seed_mult*1000+rep+offset)[0]
147         while not(terminated or truncated):
148             # take greedy action according to current policy, using theta
149             action = greedy_pi(state, theta, iht)
150             # apply action and observe system information
151             state, reward, terminated, truncated, _ = env.step(action)
152             # update episode cumulative reward
153             Gtest += reward
154             test_data[rep] = Gtest
155         mean, hw = confinterval(test_data)
156         return mean, hw
157
158 # initialize the data structure to hold the top 2Z scores and VFA parameters
159 num_best_scores = 2*Z
160 best_scores = [{'ETDR': -np.inf, 'ETDR_hw': np.inf, 'theta': None, 'iht': None} for _ in ...
161                 range(num_best_scores)]
162
163 # retain Top 2Z milestone validation testing results for final independent
164 # testing to determine superlative policy across all replications
165 def update_best_scores(mean, hw, theta, iht, best_scores):
166     # Find the first score that mean is greater than
167     for i in range(len(best_scores)):
168         if mean > best_scores[i]['ETDR']:
169             # Shift scores and parameters
170             best_scores.insert(i, {'ETDR': np.copy(mean), 'ETDR_hw': np.copy(hw), 'theta': ...
171                                   np.copy(theta), 'iht': copy.deepcopy(iht)})
172             # We only want the top scores, so remove the last one
173             best_scores.pop()
174             return True
175     return False
176
177 # record each episode's cumulative reward to measure performance
178 Gzm = [] # training results
179 GzmTest = [] # validation milestone testing results (policy evaluation)
180
181 # start timer for loading problem into memory
182 tic = time.perf_counter()
183
184 # loop through each algorithm replication for current design run
185 for z in range(Z):
186     # initialize theta vector of PFA action preference weights
187     theta = np.zeros((max_size_iht, 1))
188     # initialize integer hash table
189     iht_PFA = IHT(max_size_iht)
190     # initialize VFA weights for approximate value function
191     w = np.zeros((max_size_iht, 1))
192     # initialize integer hash table
193     iht_VFA = IHT(max_size_iht)
194     # initialize state-action visit counter
195     Csa = np.zeros((max_size_iht, 1))
196     # initialize state visit counter
197     Cs = np.zeros((max_size_iht, 1))
198
199     print(f"\n{algName} ...
200           (alpha_theta_a={alpha_theta_a:<3.2f}, alpha_theta_b={alpha_theta_b:<3.2f}) Rep {z}")
201     print(" Policy evaluation milestone validation tests...")

```

```

200
201 # loop through each episode
202 for m in range(M):
203     # initialize episode complete flag (when system enters terminal state)
204     terminated = False
205     truncated = False
206     # generate an episode following policy pi(a|s,theta)
207     # set numpy random number generator seed
208     np.random.seed(int(z*1e6+m+offset))
209     # initialize the system by resetting the environment, setting gym
210     # random number generator seed
211     # want each replication-episode combination to have its own seed
212     state = env.reset(seed=int(z*1e6+m+offset))[0]
213     # execute stochastic policy, which explores naturally
214     action = pi(state,theta,iht_PFA)
215
216     # initialize state, action, and reward queues
217     state_queue = collections.deque([])
218     action_queue = collections.deque([])
219     reward_queue = collections.deque([])
220     # full episode generation, forward loop
221     while not(terminated or truncated):
222         # record state
223         state_queue.append(state)
224         # record action
225         action_queue.append(action)
226         # apply action and observe system information
227         state, reward, terminated, truncated, _ = env.step(action)
228         # record reward
229         reward_queue.append(reward)
230         # select action via stochastic policy, which explores naturally
231         action = pi(state,theta,iht_PFA)
232
233     # initialize episode total discounted reward
234     Gm = 0
235     # REINFORCE update, backward loop
236     while len(state_queue) > 0:
237         # define state, action, reward from ends of queues
238         s = state_queue.pop() # pop() method removes last value
239         a = action_queue.pop()
240         r = reward_queue.pop()
241         # update total discounted reward
242         Gm = gamma*Gm + r
243
244         ## update theta and w vectors
245         # set target
246         Δ = Gm - Vhat(s,w,iht_VFA)
247         # determine active tiles for current state-action
248         active_tiles_x = x_indices(s,a,iht_PFA)
249         # update state-action counter
250         Csa[active_tiles_x] += 1
251         # determine active tiles for current state
252         active_tiles_phi = phi_indices(s,iht_VFA)
253         # update state counter
254         Cs[active_tiles_phi] += 1
255         # update w vector
256         w[active_tiles_phi] += alpha_w(Cs[active_tiles_phi])*Δ
257         # update theta vector
258         theta[active_tiles_x] += alpha_theta(Csa[active_tiles_x])*Δ*gamma*len(state_queue)
259
260         # compute policy pi distribution across all actions
261         current_pi = pidist(s,theta,iht_PFA)
262         # update theta vector
263         for b in range(num_actions):

```

```

264         active_tiles_b = x_indices(s,b,iht_PFA)
265         Csa[active_tiles_b] += 1 # update state-action counter
266         theta[active_tiles_b] -= ...
                current_pi[b]*alpha_theta(Csa[active_tiles_b])*Δ*gamma**len(state_queue)

267
268     # record performance
269     # print(f"\n    Episode: {m}, Cumulative reward: {Gm}")
270     Gzm.append((z,Gm))
271
272     # test current policy every test_freq episodes
273     if m % test_freq == 0:
274         mean, hw = evaluate_policy(theta, iht_PFA, num_test_reps)
275         GzmTest.append((z, m, mean, hw))
276
277         # update best scores if necessary
278         if update_best_scores(mean, hw, theta, iht_PFA, best_scores):
279             print(f"    Test... Episode: {m:>4}, EETDR CI: {mean:>6.1f} +/- {hw:4.1f} *** ...
                    New Top {num_best_scores} Policy -- theta, iht_PFA recorded")
280         else:
281             print(f"    Test... Episode: {m:>4}, EETDR CI: {mean:>6.1f} +/- {hw:4.1f}")
282
283     # last test of current algorithm replication
284     mean, hw = evaluate_policy(theta, iht_PFA, num_test_reps)
285     GzmTest.append((z,M,mean,hw))
286
287     # update best scores if necessary
288     if update_best_scores(mean, hw, theta, iht_PFA, best_scores):
289         print(f"    Test... Episode: {M:>4}, EETDR CI: {mean:>6.1f} +/- {hw:4.1f} *** New Top ...
                {num_best_scores} Policy -- theta, iht_PFA recorded")
290     else:
291         print(f"    Test... Episode: {M:>4}, EETDR CI: {mean:>6.1f} +/- {hw:4.1f}\n    ...
                *-----* Current Top 5 EETDRs: {best_scores[0]['ETDR']:>6.1f}, ...
                {best_scores[1]['ETDR']:>6.1f}, {best_scores[2]['ETDR']:>6.1f}, ...
                {best_scores[3]['ETDR']:>6.1f}, {best_scores[4]['ETDR']:>6.1f}")
292
293     # algorithm complete
294     toc = time.perf_counter()
295     print(f"\nExecuted {Z} algorithm reps in {toc - tic:0.1f} seconds.")
296
297     print(f"\nFinal testing of policies with top {num_best_scores} milestone validation test ...
            EETDRs...")
298
299     # initialize list of means and half-widths for final testing top policies
300     mean_values = []
301     hw_values = []
302
303     # loop through top policies stored in best_scores to find superlative policy
304     # via final testing with different random number seed and more reps
305     for i, score in enumerate(best_scores):
306         mean, hw = evaluate_policy(score['theta'], score['iht'], 2*num_test_reps,2)
307         print(f"    Rank: {i+1:>2d} policy final test... EETDR CI: {mean:>6.2f} +/- {hw:>4.2f}")
308         mean_values.append(mean)
309         hw_values.append(hw)
310
311     # determine superlative policy and record its mean and half-width
312     indSupETDR = np.argmax(np.array(mean_values))
313     supETDR = mean_values[indSupETDR]
314     supETDRhw = hw_values[indSupETDR]
315
316     # PFA parameter-values representing superlative policy for algorithm design run
317     theta = np.copy(best_scores[indSupETDR]['theta'])
318     iht_PFA = copy.deepcopy(best_scores[indSupETDR]['iht'])
319
320     """

```

```

321 plot algorithm performance results
322 """
323 # Organize validation milestone testing data for displaying learning curve
324 npGzmTest = np.array(GzmTest)
325 TestEETDR = np.reshape(npGzmTest[:,2], (Z,int(npGzmTest.shape[0]/Z)))
326
327 # Policy evaluation via milestone testing (validation) -- Learning Curve
328 avgTestEETDR = np.mean(TestEETDR,axis=0) # mean at each milestone
329 avgTestSE = np.std(TestEETDR,axis=0)/np.sqrt(Z)
330 avgTestHW = t.ppf(1-0.05/2, Z-1)*avgTestSE # half-width at each milestone
331
332 # Measure peak algorithm performance across multiple replications
333 maxTestEETDR = np.max(TestEETDR,axis=1)
334 meanMaxTestEETDR = np.mean(maxTestEETDR)
335 maxTestSE = np.std(maxTestEETDR,ddof=1)/np.sqrt(Z)
336 maxTestME = t.ppf(0.95, Z-1)*maxTestSE
337
338 # Measure average algorithm performance across multiple replications
339 AULC = [metrics.auc(np.arange(0,M+1,test_freq), TestEETDR[z,:])/M for z in range(Z)]
340 meanAULC = np.round(np.mean(AULC),1)
341 meAULC = t.ppf(0.95, Z-1)*np.std(AULC,ddof=1)/np.sqrt(Z)
342
343 # Algorithm score indicates both peak and average performance of hyperparameter design run
344 # Peak performance valued slightly more than average performance
345 # Reliability valued by using lower bound of single-sided confidence interval, incorporating ...
    margin of error (ME)
346 AlgScore = 0.6*(meanMaxTestEETDR-maxTestME) + 0.4*(meanAULC-meAULC)
347
348 # plot learning curve
349 plt.figure(0)
350 plt.plot(np.arange(0,M+1,test_freq),avgTestEETDR, marker = 'o', ms = 3, mec = 'k', linewidth ...
    = 1, label='Mean EETDR')
351 plt.fill_between(
352     np.arange(0,M+1,test_freq),
353     avgTestEETDR-avgTestHW,
354     avgTestEETDR+avgTestHW,
355     alpha=0.2,
356     label='95% confidence interval')
357 plt.xlabel('Episode',fontsize=9)
358 plt.ylabel('Estimated Expected\nTotal Discounted Reward (EETDR)',fontsize=9)
359 plt.title(f'{algName} Algorithm Performance, {Z} reps, {np.round((toc-tic)/(Z)/60,1)} m/rep, ...
    {envName} Gym Env\ngamma={gamma}, alpha^theta_a={alpha_theta_a:<3.2f}, ...
    alpha^theta_b={alpha_theta_b:<3.2f}, alpha^w_a={alpha_w_a:<3.2f}, ...
    alpha^w_b={alpha_w_b:<3.2f}\n Max EETDR: {meanMaxTestEETDR:>6.1f} - {maxTestME:4.1f}, ...
    Mean EETDR = {meanAULC:>6.1f} - {meAULC:4.1f}, Alg Score = {AlgScore:6.1f}\nSuperlative ...
    Policy EETDR: {supEETDR:>6.2f} +/- {supEETDRhw:4.2f}',fontsize=8)
360 plt.legend(loc="lower right",fontsize=7)
361 plt.xlim([-0.05*M, 1.05*M])
362 plt.ylim([-5, 505])
363 plt.grid(which='both')
364 plt.show()

```